

Technical Summary

1. Technical Summary — Internal Notes for Future Pipeline Design

Phase 2 — Block 1

FINAL PIPELINE ARCHITECTURE DESIGN (Databricks-Oriented Revision)

Pipeline architecture foundations

- Pipeline architecture was revised from:
local Windows-based PySpark execution
toward:
Databricks-managed distributed PySpark execution.
- The project architecture now standardizes around:
Databricks + ADLS + Airflow.
- Databricks formalized as:
the distributed computation layer
responsible for scalable PySpark execution.
- ADLS remains:
the centralized cloud persistence layer
across all pipeline stages.
- Airflow remains:
the orchestration layer
responsible for workflow automation and dependency management.

Execution architecture

- Pipeline execution order remains standardized around:
ingestion →
preprocessing →
feature engineering →
training →
evaluation →
persistence.
- Distributed processing responsibilities now execute inside:
Databricks-managed Spark environments
rather than local Spark/Hadoop environments.
- Pipeline design now reduces dependency on:
local Windows Hadoop compatibility layers.

Module responsibilities

The following production modules remain formally defined:

- `extract_kaggle.py`

- adls_io.py
- preprocess.py
- feature_engineering.py
- train_model.py
- evaluate_model.py
- fraud_pipeline_dag.py
- Existing module architecture remains largely reusable.
- Primary architectural changes affect:
infrastructure integration,
Spark session management,
ADLS connectivity handling,
and execution environment assumptions.

Databricks integration implications

- Databricks now provides:
managed Spark clusters,
distributed execution,
native Spark infrastructure,
and simplified ADLS integration.
- Local Hadoop configuration requirements become significantly reduced.
- Future production scripts should prioritize:
Databricks-compatible PySpark execution patterns.
- Future Airflow orchestration may later trigger:
Databricks jobs,
Databricks notebooks,
or Databricks workflows.

Persistence architecture

- ADLS remains the centralized persistence backbone.
- Storage zone architecture remains unchanged:
/raw/
/processed/
/features/
/models/
/evaluation/
- Parquet persistence strategy remains fully valid under Databricks execution.

Production architecture implications

- The revised architecture now prioritizes:
cloud-native execution,
distributed scalability,
modularity,
reproducibility,
and production-oriented ML engineering practices.
- Migration toward Databricks substantially reduces:
local Spark/Hadoop operational instability.

- Existing analytical findings from Phase 1 and architectural decisions from Phase 2 remain reusable and operationally consistent.

PHASE 2 — BLOCK 2: Final Data Processing and Feature Strategy (Databricks-Oriented Revision)

Distributed preprocessing architecture

- Final preprocessing strategy was revised toward: distributed Databricks-based PySpark execution.
- Preprocessing logic remains: modular, deterministic, scalable, reproducible, and inference-consistent.
- Existing preprocessing findings from Phase 1 remain operationally valid under Databricks execution.

Numerical feature strategy

- Dataset continues to maintain: fully numerical structure with pre-existing PCA-transformed latent variables.
- Unnecessary transformations on latent PCA variables should still be avoided because: V1–V28 already encode transformed latent transactional behavior.

Duplicate handling strategy

- Duplicate handling remains operationally important because: duplicated fraud observations may artificially inflate downstream evaluation performance.
- Distributed preprocessing workflows must preserve: deterministic duplicate-removal behavior across Databricks executions.

Outlier handling philosophy

- Outlier preservation remains important because: fraud signal likely exists within extreme-value transactional regions.
- Aggressive outlier deletion remains discouraged.
- Robust preprocessing approaches remain preferable over: destructive filtering logic.

Scaling strategy

- RobustScaler remains the preferred scaling approach due to:
heavy skewness,
transactional outliers,
and variance instability.
- Scaling artifacts must remain:
persistable,
reusable,
and reproducible across distributed environments.

Transformation strategy

- Log-transformed transactional amount remains formalized through:
- Log_Amount remains the preferred production-oriented Amount representation.
- Transformation consistency must remain stable across:
Databricks execution workflows,
retraining workflows,
and inference pipelines.

Imbalance-aware strategy

- Dataset continues to maintain:
approximately 578:1 imbalance ratio.
- Distributed model-training workflows must remain:
imbalance-aware,
threshold-configurable,
and operationally reproducible.

Feature engineering logic

- Feature engineering strategy remains intentionally conservative because:
the dataset already contains PCA-transformed latent representations.
- Project-specific feature engineering remains centered around:
Log_Amount,
scaling consistency,
reproducible transformations,
and future extensibility.

Reproducibility architecture

- Reproducibility requirements remain critical for:
Databricks execution consistency,
Airflow orchestration stability,
retraining workflows,
and inference reliability.
- Persisted artifacts continue to include:
feature schemas,

fitted scalers,
transformation ordering,
and reusable preprocessing logic.

Persistence architecture

- ADLS remains the centralized persistence backbone.
- Parquet remains the standardized persistence format because:
it aligns strongly with:
distributed PySpark processing,
scalable cloud storage,
and Databricks-native workflows.

Production implementation implications

- Final preprocessing architecture now supports:
cloud-native distributed execution,
scalable feature persistence,
reproducible retraining,
and production-oriented Databricks workflows.
- Existing Phase 1 preprocessing findings remain operationally reusable under the revised architecture.
-

PHASE 2 — BLOCK 3: DATA LAKE STRUCTURE DESIGN (Databricks-Oriented Revision)

Centralized persistence architecture

- ADLS Gen2 remains formalized as:
the centralized persistence backbone
across the entire fraud detection pipeline.
- Storage architecture now explicitly supports:
distributed Databricks-based PySpark execution.
- Cloud-native persistence design remains a core architectural principle.

Distributed storage compatibility

- Data lake structure was revised to align with:
Databricks distributed processing workflows.
- Storage organization now explicitly supports:
scalable distributed persistence,
distributed parquet loading,
and reusable intermediate datasets.

- Existing storage-zone architecture remains operationally valid:
/raw/
/processed/
/features/
/models/
/evaluation/

Storage zone responsibilities

- Raw zone remains responsible for:
preserving immutable source datasets.
- Processed zone remains responsible for:
cleaned and transformed distributed datasets.
- Feature zone remains responsible for:
model-ready feature persistence
and reusable feature engineering outputs.
- Model and evaluation zones remain responsible for:
serialized artifacts,
metrics,
monitoring outputs,
and reproducible historical tracking.

Parquet persistence strategy

- Parquet remains standardized because:
it strongly aligns with:
Databricks-native distributed PySpark workloads,
scalable analytical reads,
schema preservation,
and distributed cloud persistence.
- Distributed parquet persistence now becomes:
a core architectural requirement.

Storage organization philosophy

- Independent persistence across pipeline stages remains mandatory because:
it improves:
modularity,
debugging,
retraining workflows,
reproducibility,
and intermediate recovery.
- Artifact traceability remains operationally important for:
reproducible ML engineering workflows.

Reproducibility strategy

- Reproducibility architecture remains centered around:
deterministic pipeline execution,

- artifact traceability,
reusable preprocessing components,
and reproducible retraining workflows.
- Distributed workflows executed through Databricks must remain:
operationally deterministic.

Artifact versioning strategy

- Versioned artifact persistence remains important for:
rollback support,
historical experiment tracking,
reproducible evaluations,
and model lineage management.
- Timestamped parquet outputs remain recommended for:
historical reproducibility.

Partitioning considerations

- Current dataset size does not yet require:
advanced partitioning complexity.
- However:
partition-aware storage architecture remains strategically important for:
future scalability,
distributed workloads,
and large-scale transactional processing.

Production implementation implications

- The revised storage architecture now fully supports:
cloud-native distributed persistence,
scalable Databricks workflows,
Airflow orchestration,
and modular retraining pipelines.
- Existing Phase 1 and Phase 2 persistence decisions remain operationally reusable
under the Databricks-oriented architecture.

PHASE 2 — BLOCK 4: FINAL MODELING AND EVALUATION STRATEGY (Databricks-Oriented Revision)

Distributed modeling architecture

- Modeling architecture was revised toward:
Databricks-oriented distributed execution workflows.

- Distributed preprocessing and cloud-native persistence remain formally integrated into:
the final modeling architecture.
- Existing Phase 1 modeling findings remain operationally reusable under:
Databricks-managed execution environments.

Fraud-detection modeling foundations

- Problem remains formalized as:
highly imbalanced binary classification.
- Fraud behavior continues to demonstrate:
nonlinear patterns,
interaction-heavy relationships,
and strong dependence on imbalance-aware evaluation strategies.

Selected modeling strategy

- Logistic Regression remains formalized as:
interpretable baseline benchmark model.
- Random Forest remains:
the primary production candidate
due to:
operational precision stability,
nonlinear learning capability,
and low false positive behavior.
- XGBoost and LightGBM remain:
future advanced ensemble candidates
for potential performance improvements.

Evaluation strategy

- PR-AUC remains the primary operational evaluation metric because:
it better reflects fraud detection quality under severe imbalance.
- Accuracy remains operationally unsuitable as a primary metric.
- Recall and Precision remain central operational tradeoff metrics.

Validation strategy

- Stratified train/test splitting remains mandatory to:
preserve minority-class stability.
- Validation workflows executed in distributed environments must remain:
deterministic,
reproducible,
and operationally stable.

Threshold optimization strategy

- Threshold optimization remains operationally critical because:
default probability thresholds produce substantially different fraud tradeoffs.

- Production inference workflows should remain: threshold-configurable.
- False positive operational burden remains an important production consideration.

Calibration strategy

- Calibration remains a future operational enhancement candidate for: improving probability reliability, stable threshold decisions, and production fraud scoring consistency.
- Potential future approaches remain: Platt scaling and isotonic calibration.

Imbalance-aware modeling strategy

- Dataset imbalance remains approximately: 578:1.
- Class weighting, threshold tuning, and controlled undersampling remain the primary imbalance-aware strategies.
- Distributed retraining workflows must preserve: operational reproducibility and minority-class learnability.

Model persistence architecture

- ADLS remains the centralized persistence target for: trained models, scalars, schemas, metrics, and threshold configurations.
- Persisted inference artifacts remain operationally important for: reproducible production scoring.

Explainability architecture

- Explainability strategy remains centered around: feature importance consistency, fraud interpretability, and operational trust.
- Latent variables: V14, V12, V10, V17,

- and V4
remain strong predictive fraud indicators identified during Phase 1.
- SHAP remains a future explainability extension candidate.

Production implementation implications

- Final modeling architecture now supports:
scalable distributed retraining,
reproducible cloud-native persistence,
imbalance-aware operational scoring,
and Databricks-compatible workflows.
- Existing Phase 1 modeling insights remain operationally reusable under the revised architecture.

PHASE 2 — FINAL CONCLUSIONS

Final Architecture Consolidation

Phase 2 formally transformed the exploratory findings discovered during Phase 1 into a production-oriented machine learning and data engineering architecture designed for scalable fraud detection workflows.

The architecture standardized the complete end-to-end pipeline around:

- Kaggle ingestion,
- Databricks-managed distributed PySpark execution,
- Azure Data Lake Storage Gen2 (ADLS Gen2),
- modular preprocessing and feature engineering,
- reproducible model training,
- imbalance-aware evaluation,
- and Apache Airflow orchestration.

The original local Windows-based Spark execution assumptions were revised toward a cloud-native distributed execution model after identifying operational limitations associated with local Spark/Hadoop compatibility management. The revised architecture now prioritizes:

- scalability,
 - modularity,
 - reproducibility,
 - distributed processing,
 - and production-oriented ML engineering practices.
-

Distributed Processing Architecture

Databricks was formally established as:

- the distributed computation layer,
- the PySpark execution environment,
- and the scalable processing backbone of the pipeline.

This architectural transition significantly reduced dependency on:

- local Hadoop configuration,
- Windows Spark compatibility layers,
- and unstable local distributed execution environments.

The revised architecture maintains full compatibility with:

- distributed preprocessing,
 - scalable parquet persistence,
 - distributed retraining workflows,
 - and future production-oriented orchestration pipelines.
-

Data Lake and Persistence Architecture

ADLS Gen2 remains the centralized persistence backbone across all pipeline stages.

The finalized storage architecture standardized:

- /raw/
- /processed/
- /features/
- /models/
- /evaluation/

as the core storage zones responsible for:

- raw ingestion,
- distributed preprocessing outputs,

- engineered features,
- trained models,
- and evaluation artifacts.

Parquet was formally standardized as the primary persistence format due to:

- strong PySpark compatibility,
- efficient distributed analytical reads,
- schema preservation,
- scalable cloud-native persistence,
- and Databricks-native workflow alignment.

The persistence architecture additionally formalized:

- artifact traceability,
 - version-controlled outputs,
 - reproducible retraining structures,
 - and modular intermediate recovery workflows.
-

Preprocessing and Feature Engineering Strategy

The finalized preprocessing strategy preserved the fully numerical structure of the dataset while intentionally avoiding unnecessary transformations on the existing PCA-transformed latent variables.

Key preprocessing decisions formally standardized:

- duplicate-handling reproducibility,
- conservative outlier preservation philosophy,
- RobustScaler prioritization,
- Log_Amount feature generation,
- reusable preprocessing artifacts,
- and deterministic transformation ordering.

The following transformation remained formally integrated into the production architecture:

Feature engineering strategy intentionally remained conservative because:

- the dataset already contains PCA-engineered latent representations through variables V1–V28.

The architecture nevertheless preserved future extensibility for:

- temporal features,
 - interaction-based features,
 - and advanced feature engineering experimentation.
-

Modeling and Evaluation Architecture

The finalized modeling strategy formally established the project as:

- a highly imbalanced binary classification problem requiring imbalance-aware operational evaluation.

PR-AUC was standardized as the primary operational metric due to:

- severe class imbalance,
- operational fraud-detection sensitivity,
- and stronger alignment with real-world fraud detection objectives.

Random Forest emerged as the strongest current production-oriented model candidate because of:

- strong nonlinear learning behavior,
- precision stability,
- low false positive rates,
- and favorable operational tradeoffs.

Logistic Regression remained important as:

- an interpretable baseline benchmark model.

XGBoost and LightGBM were formalized as:

- future advanced ensemble candidates.

The architecture additionally standardized:

- threshold tuning,
 - calibration extensibility,
 - explainability workflows,
 - reproducible model persistence,
 - and operational fraud-scoring configurability.
-

Reproducibility and Production Engineering Foundations

Phase 2 strongly emphasized:

- deterministic preprocessing behavior,
- reproducible distributed execution,
- reusable preprocessing components,
- modular persistence,
- and operational traceability.

The finalized architecture now supports:

- scalable retraining,
- reproducible inference,
- cloud-native persistence,
- modular workflow orchestration,
- and future production deployment expansion.

Future production scripts developed during Phase 3 should therefore remain aligned with:

- distributed Databricks execution,
- reusable modular components,
- ADLS-centered persistence,
- and Airflow-compatible orchestration design.

Strategic Outcome of Phase 2

Phase 2 successfully converted:

- exploratory analytical discoveries,
- preprocessing experimentation,
- modeling experimentation,
- and storage design concepts

into:

- a formalized distributed machine learning architecture capable of supporting:
- scalable fraud detection workflows,
- reproducible retraining,
- modular orchestration,
- and production-oriented cloud-native ML engineering practices.

Phase 3 — Block 1:

PROJECT FOLDER STRUCTURE (Databricks-Oriented Revision)

Architectural transition

- Project structure was revised from:
local PySpark-oriented execution
toward:
Databricks-compatible distributed execution.
- Existing modular project organization philosophy remains operationally valid.

Repository organization philosophy

- Repository structure continues separating:
analytical workflows
from
operational production workflows.
- Phase 1 notebooks remain:
valid analytical portfolio artifacts
and should remain preserved independently from production modules.

Distributed jobs architecture

- Jobs directory now formally targets:
Databricks-compatible distributed PySpark execution.
- Production modules remain standardized around:
ingestion,
preprocessing,
feature engineering,
training,
evaluation,
and ADLS persistence.
- Scripts should remain:
reusable,
modular,
independently testable,
and orchestration-compatible.

Local storage philosophy

- Local storage now becomes:
secondary development-oriented infrastructure.
- ADLS remains:
the centralized production persistence backbone.

- Local data and model folders now primarily support: debugging, temporary validation, and development workflows.

Configuration architecture

- Configuration strategy now prioritizes: Databricks execution settings, ADLS integration, Airflow compatibility, and reusable cloud-native execution.
- Hardcoded credentials remain prohibited.

Airflow orchestration alignment

- DAG architecture now explicitly supports: Databricks workflow triggering and distributed orchestration management.
- Airflow remains responsible for: scheduling, dependency management, retries, and workflow automation.

Cloud-native execution philosophy

- Project architecture now prioritizes: distributed execution, cloud-native persistence, scalable retraining, reproducibility, and modular ML engineering workflows.
- Dependence on local Spark/Hadoop infrastructure was significantly reduced through: Databricks-managed execution environments.

Production implementation implications

- Existing project organization decisions from earlier phases remain reusable.
- The revised structure now better aligns with: modern production ML engineering repositories, distributed cloud workflows, and scalable data engineering practices.